

SafeSpace — Full Project Documentation

SafeSpace is a MERN-stack anonymous peer-support platform built as a university project. It gives people a pseudonymous, judgment-free place to talk about personal struggles, connect with peers, read something healing, and reach verified counselors — without giving up their privacy.

This document is a complete reference for the project: what it does, how it's built, every data model and API route, the design system, and what's intentionally left as a known limitation. For day-to-day setup, see the root [README.md](#); this file goes deeper.

1. Purpose & Motivation

SafeSpace explores what a genuinely trustworthy anonymous support community could look like: no real names, no email addresses, no identifying data — just a self-chosen nickname and a password. The product is deliberately **not** a bright, dopamine-driven social feed. It follows a warm, calm "Coffee & Comfort" visual theme, and it's built to hold both sides of peer support: sharing what's hard *and* reading and sharing what's good.

2. Subsystems

1. **Anonymous User & Issue System** — pseudonymous users post about personal struggles (or good news) under one of six categories: Mental Health, Relationships, Family, Financial Stress, Work & Burnout, and Gratitude & Wins.
2. **Peer Support Community** — threaded discussions per category, with replies, upvotes, search, pagination, and reporting.
3. **Professional Connect Module** — verified counselors listed with specialties and ratings; users can book anonymous sessions, and counselors have their own portal to manage those bookings.
4. **Safety & Analytics Dashboard (admin-only)** — auto-flags posts containing risk keywords, routes them to a moderation queue, handles user/content reports, and shows aggregate, privacy-safe analytics.
5. **Healing content** — a Gratitude & Wins category, a rotating daily affirmation, and a curated "Healing Reads" article page, so the site isn't only about problems.

3. Tech Stack

Layer	Technology
Runtime	Node.js $\geq$ 18
Backend framework	Express 5
Database	MongoDB Atlas (Mongoose 9)
Auth	JWT (<code>jsonwebtoken</code>) + <code>bcryptjs</code> password hashing
Rate limiting	<code>express-rate-limit</code>
Frontend	React 19 (Vite), React Router 7, Axios
Charts	Recharts (admin analytics)
Backend tests	Jest + Supertest
Frontend tests	Vitest + React Testing Library
Dev tooling	nodemon, concurrently, Oxlint

4. Architecture & Folder Structure

```

Safe_Space/
├─ backend/
│   ├─ models/           Mongoose schemas
│   ├─ controllers/      request handlers
│   ├─ routes/           Express route definitions
│   ├─ middleware/       auth, role-checking, rate limiting
│   ├─ scripts/seed.js   realistic sample-data seeder
│   ├─ tests/            Jest + Supertest suite
│   └─ server.js         Express app (exported for tests; only binds a
│                        port / connects Mongo when run directly)
├─ client/
│   ├─ src/pages/        route-level views
│   ├─ src/pages/admin/  admin dashboard sub-views
│   ├─ src/components/   reusable UI + a hand-built icon set
│   ├─ src/context/      React context providers (auth, saved posts, theme, toasts)
│   ├─ src/utils/        pure helper functions (+ their tests)
│   └─ src/assets/       the SafeSpace logo and its cropped derivatives
├─ BUILDPLAN.md          original subsystem/design plan
├─ README.md             setup, API table, feature list
├─ CONTRIBUTING.md / CODE_OF_CONDUCT.md / LICENSE
└─ PROJECT_DOCUMENTATION.md  this file

```

The client and backend are independent Node projects; the root `package.json` just runs both together via `concurrently` for local dev.

5. Authentication & Authorization Model

There are **three separate identity types**, each with its own login and token, deliberately never mixed in the same browser session:

- **User** — pseudonym + password. JWT payload: `{ id, role }` where `role` is `user`, `moderator`, or `admin`.
- **Counselor** — email + password (counselors carry real, verifiable credentials, so they're a separate Mongoose collection from anonymous users). JWT payload: `{ id, role: 'counselor' }`.
- **Admin/Moderator** — just a `User` whose `role` field has been promoted; there's no separate admin account type.

Because both User and Counselor tokens are signed with the same `JWT_SECRET`, one `protect` middleware works for both — `adminOnly` and `counselorOnly` middleware then narrow specific routes by `role`. The frontend stores whichever session is active under different `localStorage` keys (`token / user` vs. `counselorToken / counselor`); logging into either clears the other, so a lingering session can't silently authenticate a request as the wrong identity (a real bug found and fixed during end-to-end testing — see §11).

Password recovery: since pseudonymous accounts have no email, there is no "forgot password" email flow. Instead, registration returns a one-time **recovery code** (e.g. `AB3D-9KXQ-7Z2M`), shown once in a dialog the user must acknowledge. `POST /auth/reset-password` verifies that code and issues a freshly-rotated one on every successful use, so a leaked code can't be replayed.

6. Data Models

User

Field	Type	Notes
<code>pseudonym</code>	String, unique	3–32 chars
<code>passwordHash</code>	String	bcrypt, <code>select: false</code>
<code>role</code>	enum	<code>user</code> / <code>moderator</code> / <code>admin</code>
<code>avatarId</code>	Number 0–9	index into the client's <code>AVATAR_PRESETS</code>
<code>bio</code>	String	optional, ≤160 chars
<code>recoveryCodeHash</code>	String	bcrypt, <code>select: false</code> ; password-reset only

Field	Type	Notes
<code>savedPosts</code>	[ObjectId → Post]	

Post

Field	Type	Notes
<code>author</code>	ObjectId → User	
<code>category</code>	enum	6 categories, see §2
<code>content</code>	String	≤5000 chars
<code>flagged</code>	Boolean	auto-set by the risk-keyword check
<code>status</code>	enum	<code>visible</code> / <code>under_review</code> / <code>removed</code>
<code>supporters</code>	[ObjectId → User]	backs the Support reaction

A `pre('save')` hook guarantees any flagged post lands in `under_review` status even if a future code path forgets to set it explicitly — this is the single safety-critical gate before content reaches the public feed.

Thread / Reply

`Thread` mirrors `Post` (author, category, title, body) plus `upvotes` and `upvotedBy`. `Reply` belongs to **either** a `thread` or a `post` — never both, never neither — enforced by a `pre('validate')` hook, which is what lets both posts and threads support real, persisted replies through one shared model.

Counselor

Separate collection from `User` (real credentials, not a pseudonym): `name`, `email` (unique), `passwordHash`, `specialties[]`, `credentials`, `availability` (free text), `rating` (0–5), `verified`, `role: 'counselor'`.

Booking

`user` (→ User), `counselor` (→ Counselor), `requestedTime`, `notes`, `status` (`pending` → `confirmed/cancelled` → `completed`, enforced by an explicit transition table in the controller, not just the enum).

Report

`reporter`, `targetType` (`post` / `thread` / `reply` / `user`), `targetId`, `reason`, `status` (`open` / `resolved` / `dismissed`), `resolvedBy`, `resolvedAt`.

Notification

`recipient` (→ User), `type` (`post_reply` / `thread_reply` / `booking_status`), `message` , `link` (frontend route to send the user to), `read` . Created internally by a shared `notify()` helper — never exposed as a route of its own, and never fires if the recipient is the same person who triggered it.

7. Full API Reference

See [README.md → API Routes](#) for the complete, kept-up-to-date route table (Auth, Notifications, Posts, Threads & Replies, Counselors, Bookings, Reports, Analytics). At a glance, the route groups are:

- `/auth/*` — register, login, password reset, profile, stats, replies, saved posts
- `/notifications` — list, mark-all-read
- `/posts*` — CRUD, replies, support toggle, admin moderation
- `/threads*` — CRUD, replies, upvote toggle, search + pagination, "mine/supported"
- `/counselors*` — counselor auth, public listing, admin verification
- `/bookings*` — create, admin listing, counselor's own bookings + status updates
- `/reports*` — submit, admin listing, resolve
- `/admin/analytics` — aggregate counts only, no raw personal data

8. Frontend Route Map

Path	Page	Access
<code>/</code>	Post feed (hero, daily affirmation, composer, category filters)	Public
<code>/register</code> , <code>/login</code> , <code>/forgot-password</code>	Auth pages	Public
<code>/posts/new</code>	Create post	User
<code>/threads</code> , <code>/threads/:id</code>	Thread list / detail	Public (reply/upvote needs login)
<code>/threads/new</code>	Create thread	User
<code>/inspiration</code>	Healing Reads	Public
<code>/counselors</code> , <code>/counselors/:id</code>	Counselor list / profile + booking flow	Public (booking needs login)
<code>/counselor-login</code> , <code>/counselor/bookings</code>	Counselor portal	Counselor
<code>/about</code> , <code>/contact</code> , <code>/privacy</code> , <code>/terms</code> , <code>/guidelines</code> , <code>/crisis-resources</code>	Static pages	Public

Path	Page	Access
/my-activity	My Posts / Replies / Saved / Supported	User
/profile	Avatar, bio, join date, stats	User
/admin/* (posts, reports, counselors, bookings, analytics)	Moderation dashboard	Admin
*	404	Public

9. Design System — "Coffee & Comfort"

- **Palette:** warm beige/cream surfaces, muted mocha primary, sage secondary, terracotta accent, full light/dark variants (`client/src/index.css`).
- **Typography:** Fraunces (serif) for headings, Inter for body text.
- **Icons:** a small hand-built line-icon set (`Icon.jsx`) — no emoji, no icon font, anywhere in the product.
- **Avatars:** `AVATAR_PRESETS` — 10 illustrated, non-photographic placeholder avatars users pick from in Profile, never a real photo.
- **No glassmorphism:** solid warm surfaces and soft shadows, not blur/gradient panels.
- **Logo:** cropped and color-adapted from the project's own artwork into a transparent-background mark used in the navbar, favicon, auth pages, and PWA icons.
- **Illustrations:** original, flat, hand-drawn SVGs (empty states, 404, a coffee-cup-and-plant mark for About, a success checkmark for booking confirmation) — consistent with the no-photo, no-emoji visual language.

10. Feature Highlights

- Pseudonymous registration/login with one-time recovery-code password reset.
- Six-category posts and threads, each with real backend-persisted **Support** reactions, **Saved Posts**, **Supported Discussions**, and a unified **My Replies** view across both posts and threads.
- Inline, expandable replies directly under a post (not just in Threads).
- Real-time-ish **notifications** (post/thread replies, booking status changes) with an unread indicator, polled every 30s.
- Search + real backend pagination on both the Posts feed and Threads list.
- A daily rotating **affirmation** banner and a **Healing Reads** article page — curated, original, non-user-submitted content.
- Richer **profiles**: optional bio, join date, live post/reply counts.
- Full **counselor portal**: separate login, a booking dashboard with an explicit pending → confirmed/cancelled → completed state machine, and user-facing notifications when status

changes.

- Multi-step, real **booking flow** (calendar → time slot → confirm → confirmation screen) against the `/bookings` API.
- Trust & safety: report/block menu on every post, admin moderation queue, risk-keyword auto-flagging on post creation.
- **Rate limiting** on auth and content-creation routes.
- Dark mode, accessible focus states, responsive layout (mobile hamburger nav, responsive admin sidebar).
- Installable as a **PWA** (manifest + icons; no offline service worker).
- Automated tests on both sides of the stack (see §12).

11. Safety, Privacy & Known Limitations

Safety-critical behavior (do not weaken without reason): - `containsRiskKeyword()` in `postController.js` auto-flags posts matching self-harm/suicide-related phrases on creation, forcing `status: under_review` via a model-level hook so it can't be bypassed by a future code path. - Admin routes are protected by both `protect` (valid JWT) and `adminOnly` (role check) middleware — covered by automated tests. - Analytics only ever return aggregate counts, never raw personal data.

Known limitations (by design, documented, not oversights): - **Block User** is still `localStorage` - only — no backend model, so it doesn't sync across devices or stop a blocked user from seeing you. - **Counselor booking time slots** are generic placeholders (no per-counselor schedule/calendar backend yet); `availability` and `rating` are real fields, but individual client reviews aren't implemented. - **PWA** is installable metadata only — there's no service worker, so no offline support.

Bugs found and fixed via end-to-end testing (worth noting, since they show the kind of issue that only surfaces by actually driving the app, not by reading the code): - A saved-posts API call was missing its `/auth` prefix and 404'd on every request. - A lingering counselor session token in `localStorage` could silently hijack a regular user's API requests (and vice versa) — fixed by clearing the other session's storage on login. - Confirming/cancelling/completing a booking didn't re-populate the user's pseudonym in the response, so it disappeared from the counselor dashboard after a status change. - A Mongoose `pre('save')` hook using the legacy Node-callback signature was incompatible with Mongoose 9 and crashed on every flagged post — fixed by switching to the plain synchronous hook form. - `GET /threads/:id/replies` didn't exist, so thread replies never survived a page reload.

12. Testing

```
cd backend && npm test    # Jest + Supertest
cd client && npm test      # Vitest
```

- **Backend:** auth (register/login/reset-password including recovery-code rotation and replay rejection), the risk-keyword auto-flagging gate, and admin-route access control (no token / wrong role / admin token). Runs against a dedicated `safespace_test` database on the same Atlas cluster as dev (no local `mongod` in this environment), dropped after each test file; `--runInBand` avoids parallel test files racing on it.
- **Frontend:** pure-function tests (`timeAgo` , `getTodaysAffirmation`) and component render tests (`CategoryTag` , `ErrorMessage`) via Vitest + jsdom + React Testing Library.

13. Setup

See [README.md](#) → [Setup](#) for full instructions. Short version:

```
npm install      # repo root
npm run dev      # runs backend + client together
```

Backend needs `backend/.env` with `MONGO_URI` and `JWT_SECRET` (copy from `.env.example`). Run `npm run seed` in `backend/` against a dev database for realistic sample data and printed login credentials for every seeded role (user, moderator, admin, counselor).

14. Contributing & License

See [CONTRIBUTING.md](#) for the contribution workflow and [CODE OF CONDUCT.md](#). Licensed under the [MIT License](#).